

UACC: A Chiplet-Based Auxiliary Cache for Cost-Effective Performance Scaling in Multicore Processors

Abstract

Static Random Access Memory (SRAM) scaling in advanced technology nodes is approaching stagnation, and its rising cost severely limits on-chip cache capacity in modern multicore processors. To alleviate this challenge, this work proposes the Universal Auxiliary Cache Chiplet (UACC), which leverages heterogeneous technology nodes and a partially decoupled cache hierarchy, providing 86%-147% additional high-speed memory capacity at a cost equivalent to an advanced node monolithic design. To overcome the performance degradation from die-to-die latency and bandwidth (the "Chiplet Tax"), we introduce a contextualization strategy that allocates UACC capacity only where it yields net performance benefit. Our congestion-aware 2D lookahead algorithm systematically evaluates real-time access characteristics, die-to-die contention, and workload-level utility to determine both the timing and magnitude of auxiliary capacity allocation. Experimental results show UACC improves performance by 8.92% on balanced workloads and up to 27.36% on cache-sensitive ones at same costs, without degrading baseline performance.

1 Introduction

The pervasive latency challenge in general-purpose computing is primarily attributed to the growing disparity between memory access times and compute speeds—a phenomenon known as the "memory wall." While transistor density continues to scale with successive process nodes, the area density of caches, crucial for latency hiding, has largely stagnated. Measurements [2, 3, 14] indicate that the SRAM bitcell area in 3nm technology nodes has scaled by less than 5% compared to the previous generation, with certain variants even exhibiting a density regression. As a result, the economic cost of expanding die area solely to increase cache capacity in advanced processes has become prohibitively high.

Chiplet, an advanced packaging approach that achieves heterogeneous process integration through modular assembly, offers a promising solution to this challenge. Based on our modeling and evaluation of prior work [5] (detailed in Section 2), we observe that under the same N3 process cost budget, heterogeneous integration of N5 or N7 nodes can provide an additional 86%-147% high-speed SRAM capacity and transforms the otherwise exponential cost growth curve into an approximately linear trend.

However, despite the appeal of disaggregating SRAM into a separate Chiplet as explored in several prior studies [6, 9, 11] through designs that partition the Last Level Cache (LLC) and cores into a memory die and a logic die, the additional data-transfer latency introduced by die-to-die (D2D) communication, together with non-uniform effects from VLSI physical layout, increases the critical path of the memory hierarchy and elevates the average memory access time (AMAT). This degradation of baseline performance means that even conservative workloads, which would otherwise operate comfortably within a monolithic cache capacity, can still suffer noticeable performance loss. This penalty, commonly referred to as the **Chiplet Tax** [4, 5, 10, 13], poses a fundamental challenge to the practical use of chiplet-based cache extension.

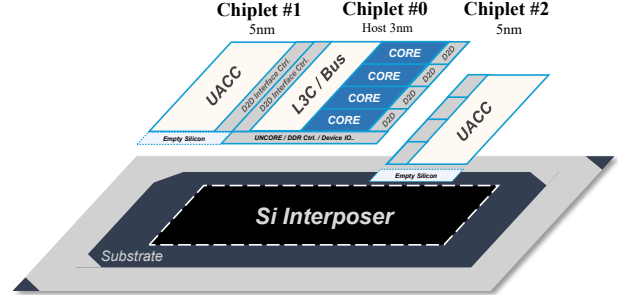


Figure 1: A 2.5D Chiplet System integrated with UACC.

To mitigate this, we adopt a *contextualization* strategy and propose the **Universal Auxiliary Cache Chiplet (UACC)**. For a multi-core host, the system is not strictly divided into a logic die and a cache die; instead, as illustrated in Figure 1, the logic portion retains a portion of the cache on chip to preserve baseline performance. Through coordinated hardware-software adaptive scheduling and control, *only* workloads that can benefit from the additional capacity, namely those with large cache demands, are allowed to offload to the UACC, while all other workloads remain on the original critical path to maintain low latency. This selective offloading mechanism defines the role of the UACC as *auxiliary*, since it supplements rather than replaces the on-chip cache and is invoked only when beneficial. By functioning in this supportive capacity, the UACC enables scalable cache expansion while preserving baseline performance.

Realizing this potential requires an allocation mechanism that can accurately identify capacity-sensitive workloads and direct them to the auxiliary cache when appropriate. Some prior studies in cache partitioning have explored dynamic allocation of a monolithic shared LLC capacity: UCP [12] allocates cache based on utility and provides practical hardware support for sampling workload capacity demand, but it lacks awareness of non-uniform cache latency; CloudCache [8] enables distance-aware reallocation of private cache capacity in a 2D mesh topology, yet its Manhattan-distance partition cannot capture congestion-induced non-uniformity stemming from limited D2D bandwidth. To address these limitations, we propose a **congestion-aware 2D lookahead** algorithm that provides effective UACC scheduling and allocation strategies for both multicore systems and configurable D2D interconnect scales.

To summarize, our main contributions are as follows:

- We propose the UACC architecture that leverages Chiplet-based heterogeneous integration to achieve cost-effective on-chip cache capacity scaling.
- We propose a hardware-software cooperative mechanism, including a congestion-aware 2D lookahead scheduling algorithm, to efficiently enable elastic cache expansion.
- We analyze Chiplet-induced non-uniformity using simulator evaluation to identify the workload and system conditions under which UACC delivers the greatest benefit.

Experimental evaluations demonstrate that UACC boosts performance by 8.92% on balanced workloads and up to 27.36% on highly cache-sensitive workloads without hurting baseline performance

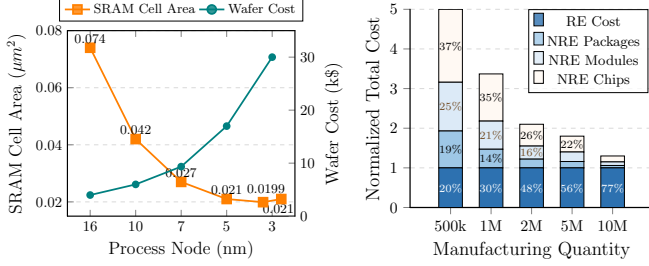


Figure 2: (a) SRAM cell area and wafer cost vs. process node; (b) RE/NRE cost breakdown vs. manufacturing quantity.

Table 1: Cost for N3 SRAM budgets and equivalent capacities.

N3 Budget	Area (mm²)	Die Cost	RE Total	Eqv. N5 Cap.	Eqv. N7 Cap.	N5-N7 Gain
1 MB	0.316	0.266	0.315	2.25MB	3.25MB	2.25×–3.25×
2 MB	0.532	0.396	0.471	4MB	5.25MB	2.00×–2.63×
4 MB	0.964	0.643	0.768	7MB	9.25MB	1.75×–2.31×
8 MB	1.828	1.115	1.339	13MB	17MB	1.63×–2.13×
16 MB	3.556	2.031	2.449	26.5MB	32.5MB	1.66×–2.03×

or incurring congestion penalties, and at essentially no additional system cost.

2 Background and Motivation

From a manufacturing cost perspective, decomposing a monolithic SoC into smaller dies significantly improves yield and reduces the effective cost per function. For a monolithic die with area A and defect density D_0 , the yield Y follows the defect-density model [5]:

$$Y \approx e^{-D_0 A}. \quad (1)$$

The recurring cost (RE) of the monolithic design can thus be expressed as:

$$C_{RE}^{mono} \propto \frac{A \cdot C_{wafer}}{Y}, \quad (2)$$

where C_{wafer} is the wafer cost per unit area. Since SRAM often occupies about 50% of the total multi-core processor area [11], integrating large SRAM arrays on advanced-node logic dies amplifies both wafer cost and yield loss. Partitioning the design into a logic die and a cache chiplet decreases each die's area, thereby improving yield and reducing the aggregate RE cost term $\sum_i \frac{A_i C_{wafer,i}}{Y_i}$.

Further cost reduction is achieved through heterogeneous integration. The logic chiplet benefits from advanced nodes for performance, while the cache chiplet can be fabricated using a mature process node with lower wafer and mask costs. The total RE cost for the heterogeneous system is then

$$C_{RE}^{chip} = \frac{A_{logic} C_{wafer}^{p_{logic}}}{Y_{logic}^{p_{logic}}} + \frac{A_{cache} C_{wafer}^{p_{cache}}}{Y_{cache}^{p_{cache}}}, \quad (3)$$

where p_{logic} and p_{cache} denote the respective process nodes. Because SRAM scaling offers diminishing density improvement in advanced technologies, this heterogeneous partition minimizes RE cost without compromising functionality.

On the non-recurring engineering (NRE) side, the cache chiplet exhibits strong design reuse across product lines. If its design and verification cost C_{NRE}^{cache} is amortized over R products, the average

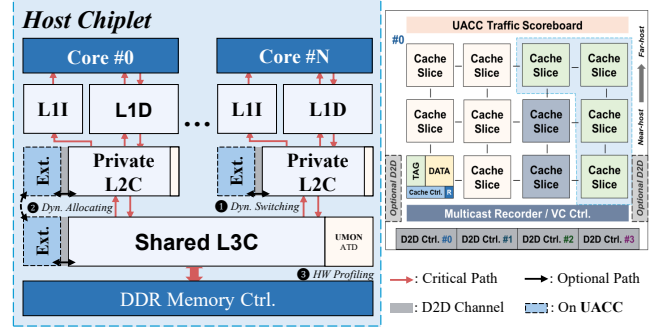


Figure 3: Chiplet-based Elastic Cache Hierarchy with 2D-Mesh Network-based UACC Hardware Architecture

contribution per design becomes C_{NRE}^{cache}/R . The total system cost can thus be written as

$$C_{chip} = \sum_i \frac{A_i C_{wafer,i}}{Y_i} + C_{NRE}^{logic} + \frac{C_{NRE}^{cache}}{R} < C_{mono}. \quad (4)$$

Figure 2(b) highlights this trend: as manufacturing volume increases, RE cost dominates while the amortized NRE cost fraction decreases substantially. This reinforces the economic advantage of reusable chiplets, especially for SRAM-heavy processors.

When the system-level bandwidth and latency constraints are satisfied, Equation (4) indicates that the chiplet partition is economically favorable whenever the yield and process-node savings outweigh the integration overhead. Therefore, isolating the cache as a reusable chiplet becomes a cost-effective architectural choice for multicore processors with high SRAM ratios. As shown in Table 1, a chiplet built in an N5 process offers on average 1.86× more SRAM capacity at equal silicon cost, and an N7 chiplet provides 2.47× more capacity. In practical designs, these gains can be conservatively approximated as 1.5× and 2× effective capacity scaling, respectively.

3 UACC Architecture

As depicted in Figure 3, the UACC functions as an on-demand auxiliary layer that is activated through hardware sampling, dynamic scheduling, and adaptive allocation mechanisms. This design preserves the latency-critical cache hierarchy of the baseline processor while enabling elastic capacity scaling through chiplet integration.

Notably, due to D2D bandwidth limits and transmission latency, cache ways expanded in UACC exhibit Non-Uniform Cache Access (NUCA) between local and remote chiplet. In fact, Chiplet-based cache hierarchies follow two main directions: Uniform Cache Access (UCA) versus NUCA, corresponding to the two UACC integration approaches in Figure 1: extending the shared cache layer (L3/LLC, #1, UCA) or the private cache layer (L2, #2, NUCA). The UCA-style static capacity expansion is more practical from an engineering standpoint, such as 3D-stack, requiring minimal microarchitectural modification and enabling a more deterministic physical layout, thus widely adopted. NUCA may appear as a naive alternative. However, as Chiplet integration enables rapid cache capacity growth, compounded long-path latency and unpruned traffic quickly saturate D2D bandwidth, causing inter-die congestion and pushing the cache system toward its hierarchical latency bound. Under such scaling, NUCA is no longer optional but necessary. Therefore, this work focuses on the private-layer extension (#2), a NUCA-based approach, to relax this bound.

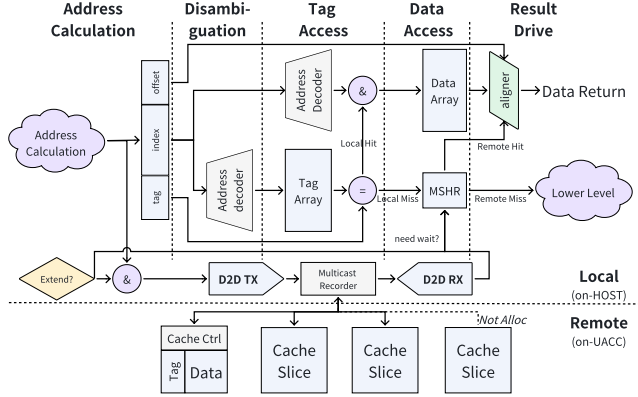


Figure 4: Pipelined way extension for UACC.

Pipelined Integration. As illustrated in Figure 4, the extended pipeline enables parallel HOST and UACC tag lookups, with Miss Status Handling Register support for out-of-order transaction; upon an remote hit and local miss, a **swap operation** atomically exchanges the accessed line with a local victim (e.g., LRU), thereby promoting frequently accessed data into the local cache and maintaining its contents in a near-optimal LRU order.

Multicast Handling. To minimize host-side modifications and suppress unnecessary cross-die traffic, the UACC implements a **multicast recorder** that performs parallel tag probes across its internal slices. When the host issues a remote lookup, the recorder aggregates slice responses locally within the UACC, ensuring that only the final hit/miss outcome is returned over the D2D link. This design maintains the existing on-host organization, preventing a microarchitectural redesign and additional D2D communication.

Coherence Compatibility. Because the UACC functions solely as a private-cache way extension rather than an independent coherence participant, the system’s coherence protocol remains unchanged. Remote tag access follows the same pipelined lookup structure described earlier, simply adding a remote way check without introducing new coherence states or messages. As a result, coherence behavior stays fully transparent while the UACC provides elastic capacity augmentation.

4 Elastic Expansion Scheduling

4.1 Cache Sensitivity

To understand when elastic capacity extension is beneficial, we first characterize the capacity sensitivity of representative SPEC CPU2017 workloads. For each workload, we compute

$$\text{Speedup} = \frac{\text{IPC}(C_i)}{\text{IPC}(256 \text{ KB})}, \quad (5)$$

where C_i denotes an enlarged private-cache configuration. For clarity of analysis, all metrics are derived from demand misses only, excluding prefetch-generated accesses. As shown in Figure 5, Capacity-sensitive workloads exhibit speedup ratios significantly exceeding 1.0, indicating working sets surpassing the baseline cache size and thus benefiting markedly from added cache ways. In contrast, capacity-insensitive workloads show speedup tightly clustered around 1.0 across all cache capacities—either fitting comfortably within 256 KB or being compute- or stream-bound, rendering additional cache capacity ineffective. This separation provides a clear and quantifiable

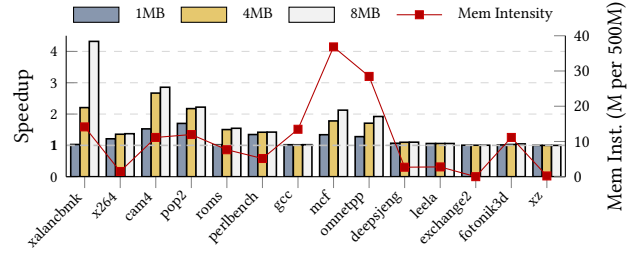


Figure 5: Cache capacity sensitivity and memory intensity of SPEC CPU2017 workloads under configs in Table 2.

basis for deciding when extended capacity has meaningful performance impact.

Orthogonally, Figure 5 characterizes memory-access intensity in different workloads. In D2D interconnects, data are transmitted in flow control units (flits), which are temporarily stored in input-port or virtual-channel buffers before being injected into the link. Under light load, flits experience nearly constant latency dominated by propagation and routing delays. However, as cross-chiplet cache traffic increases and the D2D link approaches its available bandwidth, newly arrived flits frequently encounter occupied transmit ports and must queue as buffered flits. When examined per D2D direction, the superposition of many independent access streams approximates a Poisson arrival process, while the service time depends on packet or flit sizes and each link behaves as a single server. Therefore, this behavior can be captured using an M/G/1 queue model [15]. By the Pollaczek-Khinchine result [15], the average queuing delay grows approximately as $1/(1-\rho)$, where ρ is link utilization, implying a sharp escalation in end-to-end access latency as approaching saturation ($\rho \rightarrow 1$). Once remote cache traffic pushes the D2D link into the high utilization regime, queuing dominates latency instead of propagation, causing actual remote-access costs to deviate substantially from the constant-latency assumptions used in the software scheduler. This divergence can systematically invalidate performance model based task placement unless D2D bandwidth and queuing effects are explicitly accounted for.

Both workload characteristics underpin our elastic-capacity decision policy at the private cache level. We treat the local private cache and its UACC extension as a single logical private cache, and model performance using $\text{AMAT} = \text{HitTime} + \text{MissRate} \times \text{MissCost}$ where MissRate denotes the fraction of first-level accesses that miss in this unified private cache, and MissCost is the average service latency of such misses (e.g., to lower level cache or main memory). Comparing the operation enable and disable UACC yields:

$$\begin{aligned} \Delta \text{AMAT} &= \text{AMAT}_{\text{Enable}} - \text{AMAT}_{\text{Disable}} \\ &= \Delta \text{HitTime} + \Delta \text{MissRate} \times \text{MissCost}, \end{aligned} \quad (6)$$

where $\Delta \text{HitTime} = \text{HitTime}_{\text{Enable}} - \text{HitTime}_{\text{Disable}}$ reflects the increased average hit latency due to remote UACC ways accessed across chiplets (including D2D latency and potential congestion), while $\Delta \text{MissRate} = \text{MissRate}_{\text{Enable}} - \text{MissRate}_{\text{Disable}} < 0$ captures the benefit of higher effective capacity. Although D2D traversal inevitably increases hit latency ($\Delta \text{HitTime} > 0$), the capacity expansion simultaneously reduces miss rate ($\Delta \text{MissRate} < 0$), thereby avoiding costly accesses to lower-level memory. UACC activation is beneficial whenever $\Delta \text{AMAT} < 0$, i.e., when the MissRate reduction

outweighs the HitTime penalty. Our controller evaluates this condition online to dynamically enable or disable the extended capacity.

4.2 Congestion-aware Scheduling

Based on the preceding analysis, the scheduling policy algorithm for the controller is presented for balancing benefit and penalty. Each core i possesses a fixed local cache capacity represented by a base LRU depth Pos_{base} ; any additional ways assigned to that core occupy deeper recency positions and are served by remote banks. The allocation vector $\mathbf{k} = (k_1, \dots, k_N)$ determines the number of remote ways granted to each core. Increasing k_i reduces the core's residual miss rate but simultaneously increases D2D traffic by converting more L1 misses into remote hits.

To quantify per-core behavior under any allocation k , we adopt UCP [12]'s auxiliary tag directory (ATD) and utility monitor. For every L1 miss from core i , the associated ATD records a hit at recency position r and increments counter $H_i[r]$. At the end of a profiling time window T , these counters represent the distribution of reuse distances under LRU. Given a fixed local depth Pos_{base} , the first Pos_{base} positions correspond to local capacity and the next k positions to remote. The residual, or bad, miss rate under allocation k is given by:

$$\text{miss}_k(i) = \frac{N_i^{L1\text{miss}} - \sum_{r=0}^{\text{Pos}_{\text{base}}+k-1} H_i[r]}{N_i^{L1\text{miss}}}. \quad (7)$$

Likewise, the number of remote hits is $\sum_{r=\text{Pos}_{\text{base}}}^{\text{Pos}_{\text{base}}+k-1} H_i[r]$, yielding the remote-hit probability:

$$p_i(k) = \frac{\sum_{r=\text{Pos}_{\text{base}}}^{\text{Pos}_{\text{base}}+k-1} H_i[r]}{N_i^{L1\text{miss}}}. \quad (8)$$

These two quantities expose a structural conflict: deeper positions reduce residual misses but also amplify remote traffic across D2D.

We incorporate this conflict into a marginal evaluation metric. The per-step capacity gain for extending allocation $k \rightarrow k+1$ is:

$$G_i(k) = (\text{miss}_k(i) - \text{miss}_{k+1}(i)) \cdot \delta(d_{i,k+1}), \quad (9)$$

where $\delta(d_{i,k+1})$ applies Manhattan distance-dependent discount weighting, related to specific hardware implementation and topology. The traffic increment induced by gaining an additional remote way is:

$$\Delta\lambda_i(k) = m_i(p_i(k+1) - p_i(k))c_{\text{resp}}, \quad (10)$$

which increases the interconnect arrival rate. Here, $m_i = N_i^{L1\text{miss}}/T$ represents the L1 miss generation rate, c_{resp} represents the per-response traffic unit on the D2D link, commonly 1 flit for a full cacheline transfer in typical implementations. And the queuing delay follows the M/G/1 approximation:

$$P_i^{\text{cong}}(k) = W(\lambda + \Delta\lambda_i(k)) - W(\lambda) \quad (11)$$

with

$$W(\lambda) = \frac{\lambda RTT^2}{2(1 - \lambda RTT)}. \quad (12)$$

The marginal utility combines both effects:

$$MU_i(k) = G_i(k) - P_i^{\text{cong}}(k), \quad (13)$$

and determines whether a step is beneficial in a global sense.

Algorithm Implementation. The allocator operates through an incremental but lookahead-enabled procedure described in Algorithm 1. It initializes all $k_i = 0$ and computes the initial arrival

Algorithm 1 Congestion-Aware Lookahead Allocation

```

1: Initialize  $k_i \leftarrow 0$  for all  $i$ ; compute initial  $\lambda$ 
2: while true do
3:    $\mathcal{U} \leftarrow \emptyset$ 
4:   for each core  $i$  do
5:     for each  $\Delta \in \mathcal{D}$  with  $k_i + \Delta \leq k_i^{\text{max}}$  do
6:       Compute  $G_i(k_i \rightarrow k_i + \Delta)$ 
7:       Compute  $\Delta\lambda_i(k_i \rightarrow k_i + \Delta)$ 
8:        $P_i^{\text{cong}} = W(\lambda + \Delta\lambda_i) - W(\lambda)$ 
9:        $MU_i = G_i - P_i^{\text{cong}}$ ; add  $(i, \Delta, MU_i)$  to  $\mathcal{U}$ 
10:    end for
11:  end for
12:   $(i^*, \Delta^*) = \arg \max_{(i, \Delta) \in \mathcal{U}} MU_i$ 
13:  if  $MU_{i^*, \Delta^*} \leq 0$  then
14:    break
15:  end if
16:   $k_{i^*} \leftarrow k_{i^*} + 1$ ;  $\lambda \leftarrow \lambda + \Delta\lambda_{i^*, 1}$ 
17:  Recompute congestion penalties for remaining entries
18: end while
19: return  $\mathbf{k}^* = (k_1, \dots, k_N)$ 

```

rate. At each iteration, it forms a set of feasible expansion candidates by accumulating multi-way utility entries $MU_i(k_i \rightarrow k_i + \Delta)$ over predefined increments Δ , enabling lookahead across future recency positions. The allocator then selects the entry with the highest utility and commits only a single-way increase $k_{i^*} \leftarrow k_{i^*} + 1$, thereby keeping the allocation incremental while preserving the predictive value of multi-way evaluation. After each commitment, the global arrival rate is updated and all remaining utilities recompute only their congestion-penalty terms, reflecting the new queuing conditions while reusing previously computed capacity gains. The process continues as long as the chosen utility remains positive; otherwise it terminates. The resulting allocation vector $\mathbf{k}^*$ retains only steps whose benefits outweigh their congestion costs. Since residual-miss reduction exhibits diminishing returns and the M/G/1 queuing delay escalates rapidly near saturation, the allocator naturally halts before remote capacity becomes detrimental.

Hardware Profiler The resulting policy is lightweight to implement, requiring only per-core utility monitor with 1920 bytes of storage for 32 sampled sets, which amounts to under 0.2% of a 1 MB cache.

5 Evaluation

In this section, we evaluate UACC with two goals: (a) to show that chiplet-based capacity extension can improve performance under a cost-equivalent cache budget, and (b) to demonstrate that our congestion-aware allocation algorithm is effective under diverse workload pressures and D2D interconnect tiers. We summarize the benchmark suite, simulation settings, and baseline designs below.

5.1 Experiment Setup

Benchmarks. To evaluate the proposed mechanisms under controllable cache-pressure scenarios, we construct a multi-program workload suite from the memory access trace of SPEC CPU2017 [1]. We classify workloads by cache-capacity sensitivity into three groups: **high** {x1anCBMK, cam4, pop2}, **medium** {x264, roms, perlbench}, and **low** {all others} (mcf and omnetpp are included here due to

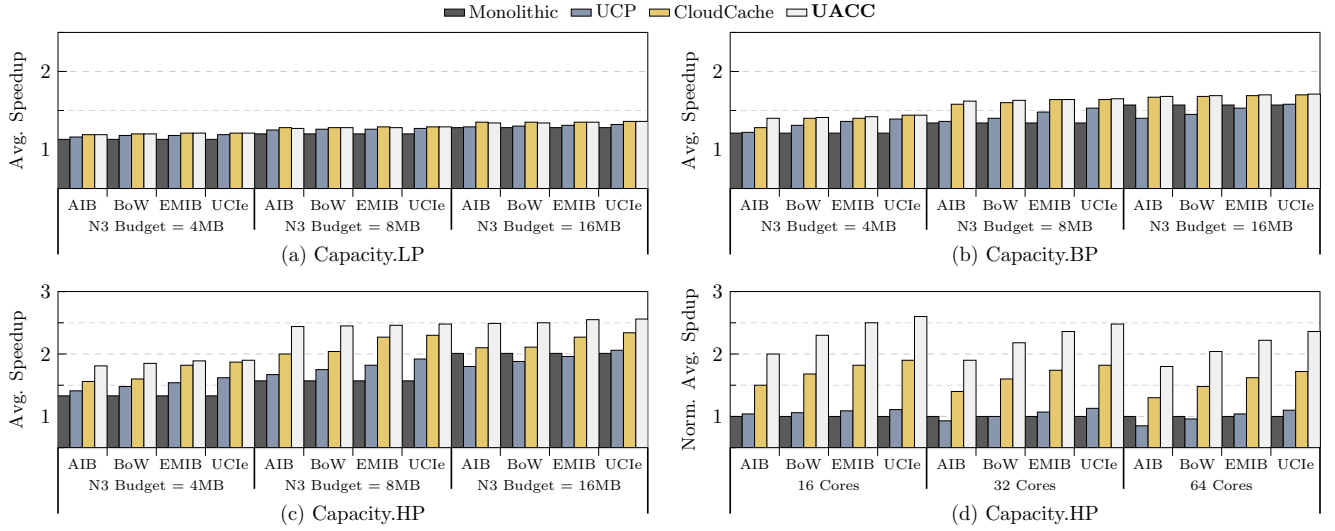


Figure 6: Overall performance comparison between UACC and baseline cache designs under different cache sizes.

Table 2: ChampSim Simulation Configuration

Component	Configuration
Core	4 OoO core @ 4.0 GHz; ROB = 352; Scheduler = 128; 6-wide fetch/decode/dispatch, 4-wide issue, 5-wide retire; LQ/SQ = 128/72 (2 entries/cycle)
Branch Predictor	Bimodal + basic BTB; Mispredict penalty = 1 cycle
L1I	32 KiB, 8-way, 64B lines; MSHR = 8; Lat = 4 cycles; RQ/WQ/PQ = 64/64/32; next-line prefetching
L1D	48 KiB, 12-way, 64B lines; MSHR = 16; Lat = 5 cycles; RQ/WQ/PQ = 64/64/8; next-line prefetching
L2C	Private, 256 KiB, 16-way; MSHR = 32; Lat = 10 cycles; RQ/WQ/PQ = 32/32/16
TLB	ITLB/DTLB = 64 entries each; STLB = 1536 entries; Hit latencies = 1 / 1 / 8 cycles
Page Table	5-level, 4 KiB pages; Minor fault = 200 cycles
Memory	DDR-3200 equivalent; tCAS/tRCD/tRP/tRAS = 24/24/24/52; 1-channel;

Table 3: Evaluated D2D Interfaces for UACC Integration

Interface	RTT (ns)	Bandwidth (GB/s)	Buffer Dep. (flits)	Norm. Area (mm ² /Tb/s)	Cost (\$)
AIB	10–15	10–120	8–16	0.6–0.8	Low
BoW	8–12	50–250	16–32	≈0.8	Low-Med
EMIB	6–10	100–300	24–32	≈0.8	Medium
UCle	4–8	128–512	24–64	≈0.31	High

their very small absolute gains). For multi-core experiments, mixes are generated by sampling from these groups.

We form three representative workload class: Low-Pressure (LP) with 0% high-, 30% medium-, and 70% low-sensitivity programs; Balanced-Pressure (BP) with 33% high-, 33% medium-, and 34% low-sensitivity programs; and High-Pressure (HP) with 90% high-, 9% medium-, and 1% low-sensitivity programs. Each class is evaluated by randomly assigning a workload to each core (with replacement) and repeating this process five times per class to report the average.

Configuration. We evaluate UACC using a ChampSim [7]-based

traced simulation with 4 out-of-order cores, whose microarchitectural parameters are listed in Table 2. Each core runs one workload selected from mixed sets, with a warmup of 1 million instructions followed by 500 million instructions for measurement. For the distance-discount $\delta(d_{i,k+1})$, we set on-HOST cache accesses to 100%, apply a 0.53% reduction per 1ns latency (equal to 1 router hop).

Then we implement a cost-equivalent design-space exploration that incrementally shifts LLC capacity from the monolithic N3 host die to a remote chiplet fabricated in a mature node. Starting from an N3 monolithic LLC budget of 4 MB, 8 MB, or 16 MB, we initialize the UACC chiplet with zero capacity and progressively shift cache ways from the host die to the chiplet. In each step, we subtract 256 KB (4 slices $\times$ 64 KB) from the on-HOST LLC and add the same amount to on-UACC. Through a sweep of configurations ranging from fully monolithic to increasingly chiplet-augmented caches, we quantify the additional performance that UACC recovers or improves under an equivalent silicon cost, using average speedup (5) as the metric. Cost equivalence is maintained using per-MB SRAM area and per-mm² process-technology cost models [5], ensuring that all compared configurations correspond to identical N3-normalized budgets.

We further sweep four D2D interface tiers of increasing integration capability, as summarized in Table 3, by varying link bandwidth and round-trip latency while keeping the routing topology and buffer depth fixed. To create additional traffic sources and intensify congestion pressure, we also evaluate larger multicore configurations with 16, 32, and 64 cores, respectively, together with 2 MB budget per core, where runtime contention and queue buildup arise naturally during execution.

Baselines. We compare UACC against four representative cache allocation strategies, yielding an *ablation* study: (1) Monolithic-Static employs a monolithic N3 LLC without way extension, representing the cost-equivalent baseline. (2) UCP [12]-Greedy implements utility-based capacity allocation using marginal miss reduction, but without distance-awareness, and all capacity is exposed through a unified address space, isolating the benefit of dynamic capacity management. (3) CloudCache [8] adopts dynamic global partitioning combined with distance-aware allocation, but remains agnostic to network

congestion. (4) UACC (our design) extends distance-aware allocation by incorporating congestion-aware utility adjustment.

5.2 Results and Analysis

Figure 6 summarizes average or normalized system speedup across three N3-equivalent capacity budgets (4/8/16 MB), four D2D tiers (AIB, BoW, EMIB, UCIE), and three congestion-pressure regimes. The left column reports average speedup for *Capacity*.{LP, BP, HP} mixes, while the right column fixes *Capacity*.HP and normalizes each dynamic scheme to the corresponding **Monolithic-Static** design as we scale core counts from 16 to 64. Across all settings, **UACC** (distance + congestion aware) matches or outperforms **Monolithic-Static**, **UCP-Greedy**, and **CloudCache** (distance aware).

5.2.1 Impact of capacity budget. Under *Capacity*.LP, all four schemes are nearly indistinguishable: average speedup ranges from ≈ 1.13 – $1.21\times$ at 4 MB to ≈ 1.28 – $1.36\times$ at 16 MB, and **UACC** tracks **CloudCache** within 1–2% across all D2D tiers while modestly improving over **Monolithic-Static** (up to $\approx 6\%$). This indicates that UACC behaves correctly: in well-cached scenarios, it avoids harming baseline performance and introduces virtually no additional overhead.

As pressure increases, the budget becomes more critical. Under *Capacity*.BP, **UACC** consistently improves upon both **UCP-Greedy** and **CloudCache**. For instance, at 4 MB with AIB, UACC reaches $1.40\times$ versus $1.28\times$ for CloudCache and $1.22\times$ for UCP, i.e., roughly +9% and +15% higher, respectively. At 8–16 MB, UACC closely tracks CloudCache (within $\approx 1\%$) but continues to eliminate UCP’s regressions, indicating that congestion-aware capacity steering mainly protects against pathological placements rather than dramatically changing the average in the moderate-pressure regime.

The effect is most pronounced under *Capacity*.HP. With heavy cache pressure, scaling the budget from 4→8 MB substantially amplifies the benefit of placing capacity near the host and carefully throttling remote traffic. For example, with EMIB, UACC improves from $1.89\times$ to $2.46\times$ as the budget increases from 4 MB to 8 MB, while Monolithic only rises from $1.33\times$ to $1.57\times$. At 8 MB, UACC delivers up to $2.46\times$ speedup, outperforming UCP ($1.82\times$) and CloudCache ($2.27\times$) by about 35% and 8%, respectively. At 16 MB, improvements taper: Monolithic already reaches $\approx 2.0\times$, and UACC’s speedup saturates around 2.5 – $2.6\times$. The additional remote capacity remains beneficial (UACC still provides $\approx 27\%$ over Monolithic and ≈ 10 – 12% over CloudCache), but with diminishing marginal utility once the system becomes less capacity-bound.

5.2.2 Impact of D2D tier (distance/capability). Sweeping from **AIB** → **UCIE** monotonically improves all dynamic schemes by raising bandwidth and lowering RTT. Under *Capacity*.LP and *Capacity*.BP, this manifests as a gentle upward trend: for example, at 8 MB under *Capacity*.BP, UACC moves from $\approx 1.62\times$ (AIB) to $\approx 1.65\times$ (UCIE), maintaining a small but consistent edge over CloudCache (1.58 – $1.70\times$) and UCP (1.36 – $1.58\times$). These configurations are not strongly congestion-limited, so distance awareness dominates, and UACC behaves similarly to a well-tuned distance-aware scheme.

In contrast, under *Capacity*.HP the D2D tier interacts strongly with congestion. At 8 MB, UACC delivers ≈ 2.44 – $2.48\times$ speedup across AIB–UCIE, while CloudCache achieves ≈ 2.00 – $2.30\times$ and UCP only ≈ 1.67 – $1.92\times$. Upgrading the D2D fabric raises everyone’s ceiling, but UACC converts a larger fraction of that raw capability into effective

performance because it explicitly prices both distance and queuing delay into its marginal-utility decisions. The result is that UACC retains a healthy lead even on the strongest fabric (UCIE), instead of letting extra bandwidth simply mask poor placement choices.

5.2.3 Congestion stress via core scaling. The right column of Figure 6 stresses congestion by fixing *Capacity*.HP and scaling core counts (16/32/64 cores), normalizing each bar to its Monolithic baseline. **UCP-Greedy** becomes brittle as contention grows: for weaker tiers (AIB/BoW), its normalized speedup drops below 1.0 at 32 and 64 cores (e.g., down to $0.85\times$ for 64-core AIB), indicating that aggressively pushing traffic to remote capacity can be worse than staying entirely local. **CloudCache** is more robust, maintaining ≈ 1.3 – $1.9\times$ speedup, but its gains flatten as links become saturated and distance-only decisions fail to avoid hot spots.

UACC, in contrast, scales gracefully. Across all tiers and core counts, UACC delivers ≈ 1.8 – $2.6\times$ speedup over Monolithic, and its advantage over CloudCache widens under heavier congestion. For example, at 16 cores with UCIE, UACC reaches $2.60\times$ vs. $1.90\times$ for CloudCache and $1.11\times$ for UCP (about +37% and $> 2.3\times$ higher, respectively). At 64 cores with AIB, UACC still provides $1.80\times$ speedup, where CloudCache achieves $1.30\times$ and UCP only $0.85\times$ (roughly +38% over CloudCache and $> 2\times$ over UCP). These results show that explicitly modeling congestion allows UACC to preserve and even amplify its benefits as the system scales, rather than collapsing back to (or below) Monolithic performance.

5.2.4 Takeaways. (i) **Congestion-aware capacity steering** is the dominant factor once remote traffic becomes material: distance-aware placement (CloudCache) is necessary but not sufficient to avoid D2D bottlenecks, especially at high pressure and high core counts. (ii) Benefits peak at **moderate budgets (around 8 MB)** where both capacity pressure and remote traffic are significant; at low budgets, the system remains bandwidth/latency limited, while at high budgets, marginal utility diminishes as working sets already fit. (iii) Upgrading the **D2D tier** benefits all schemes, yet **UACC** achieves the largest gains, using higher bandwidth and lower RTT to reduce remote round-trips and improve remote hit rates even under severe congestion. These results confirm UACC’s design goal: to convert heterogeneous capacity into effective performance under real congestion rather than under idealized distance assumptions.

6 Conclusion

In this work, we presented UACC, a chiplet-based auxiliary cache architecture that exploits heterogeneous integration to deliver cost-effective capacity scaling for multicore processors. By treating remote cache capacity as an on-demand extension of the private cache hierarchy rather than a wholesale replacement, UACC preserves the latency-critical baseline path while selectively offloading only capacity-sensitive workloads. Our congestion-aware 2D lookahead allocator coordinates per-core utility sampling with an explicit model of D2D queuing delay, enabling the system to steer traffic toward remote capacity only when the reduction in residual miss rate outweighs the additional inter-chiplet latency. Our simulations across varied cache budgets, D2D interfaces, and workload mixes show that UACC consistently outperforms monolithic baselines and prior utility- or distance-based partitioning schemes, with the largest gains under higher cache pressure and noticeable interconnect contention.

References

- [1] James Bucek, Klaus-Dieter Lange, and Jóakim v. Kistowski. 2018. SPEC CPU2017: Next-generation compute benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 41–42.
- [2] Jonathan Chang, Yen-Huei Chen, Gary Chan, Hank Cheng, Po-Sheng Wang, Yangsyu Lin, Hidehiro Fujiwara, Robin Lee, Hung-Jen Liao, Ping-Wei Wang, Geoffrey Yeap, and Quincy Li. 2020. 15.1 A 5nm 135Mb SRAM in EUV and High-Mobility-Channel FinFET Technology with Metal Coupling and Charge-Sharing Write-Assist Circuitry Schemes for High-Density and Low-VMIN Applications. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*. 238–240. <https://doi.org/10.1109/ISSCC19947.2020.9062967>
- [3] Jonathan Chang, Yen-Huei Chen, Wei-Min Chan, Sahil Preet Singh, Hank Cheng, Hidehiro Fujiwara, Jih-Yu Lin, Kao-Cheng Lin, John Hung, Robin Lee, Hung-Jen Liao, Jhon-Jhy Liaw, Quincy Li, Chih-Yung Lin, Mu-Chi Chiang, and Shien-Yang Wu. 2017. 12.1 A 7nm 256Mb SRAM in high-k metal-gate FinFET technology with write-assist circuitry for low-VMIN applications. In *2017 IEEE International Solid-State Circuits Conference (ISSCC)*. 206–207. <https://doi.org/10.1109/ISSCC.2017.7870333>
- [4] Pete Ehrett, Todd Austin, and Valeria Bertacco. 2021. Chopin: Composing cost-effective custom chips with algorithmic chiplets. In *2021 IEEE 39th International Conference on Computer Design (ICCD)*. IEEE, 395–399.
- [5] Yinxiao Feng and Kaisheng Ma. 2022. Chiplet actuary: A quantitative cost model and multi-chiplet architecture exploration. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 121–126.
- [6] Pouya Fotouhi, Sebastian Werner, Jason Lowe-Power, and SJ Ben Yoo. 2019. Enabling scalable chiplet-based uniform memory architectures with silicon photonics. In *Proceedings of the International Symposium on Memory Systems*. 222–234.
- [7] Nathan Gober, Gino Chacon, Lei Wang, Paul V Gratz, Daniel A Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324* (2022).
- [8] Hyunjin Lee, Sangyeun Cho, and Bruce R Childers. 2011. CloudCache: Expanding and shrinking private caches. In *2011 IEEE 17th International Symposium on High Performance Computer Architecture*. IEEE, 219–230.
- [9] Fuping Li, Ying Wang, Yujie Wang, Mengdi Wang, Yinhe Han, Huawei Li, and Xiaowei Li. 2024. Chipletizer: Repartitioning socs for cost-effective chiplet integration. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 58–64.
- [10] Tao Li, Jie Hou, Jinli Yan, Rulin Liu, Hui Yang, and Zhigang Sun. 2020. Chiplet heterogeneous integration technology—Status and challenges. *Electronics* 9, 4 (2020), 670.
- [11] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H Loh, Mahesh Subramony, and Sean White. 2021. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 57–70.
- [12] Moinuddin K Qureshi and Yale N Patt. 2006. Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches. In *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*. IEEE, 423–432.
- [13] Dylan Stow, Itir Akgun, and Yuan Xie. 2019. Investigation of cost-optimal network-on-chip for passive and active interposer systems. In *2019 ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP)*. IEEE, 1–8.
- [14] Xiaofei Wang, Yusung Kim, Gwang Hyeon Baek, Kunal Girish Bannore, Kaushal Dave, Arash Joushaghani, Narae Kang, Minwoo Ko, Anandkumar Mahadevan Pillai, Hema Chandra Prakash Movva, Gysung Park, Mukhtadir Rahman, Seenivasan Subramaniam, Vinay Vashishtha, Teng Yang, Zheng Guo, and Eric A Karl. 2025. 29.2 A 0.021 μm^2 High-Density SRAM in Intel-18A-RibbonFET Technology with PowerVia-Backside Power Delivery. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 68. 494–496. <https://doi.org/10.1109/ISSCC49661.2025.10904657>
- [15] John A White. 2012. *Analysis of queueing systems*. Elsevier.